MOD -03 - JDK new Features

MOD -03 - JDK new Features

LUMINAR
TECHNOLAB

Method Reference in Java 8

Method Reference is a wonderful feature introduced in Java 8.

one of the biggest advantage in using a Lambda expression is to minimize the lines of code. Similarly, Method reference also minimizes lines of code even more than Lambda. However, we use both of them in the presence of functional interfaces only. Moreover, It is also a concise and simpler form of lambda expression.

- ✓ Method reference is a new feature of Java 8, a short and easily readable writing syntax for a lambda expression.
- ✓ To refer to a functional interface method, use a method reference.
- ✓ You can replace a method reference for your lambda expression whenever you are just referring to a method with a lambda expression.

we use lambda expressions to implement Functional interfaces with minimum lines of code and even to get better code readability. Similarly, we can use Method References(::) Java 8 to implement Functional interfaces with even lesser code again than lambda expressions and this time we get the benefit of code re-usability as well, because we don't provide implementation for functional interface. Instead, we provide reference to already existing method(with similar argument types) to simplify the implementation of the functional interface using double colon (::) operator. **This process of providing reference to pre-existing method is called Method reference.**

while applying Method References our focus is always on the pre-existing methods which are well suited to the implementation of Functional Interface. In fact, pre-existing methods should have same arguments in number & type and there are no restrictions on their return type as such.

In method reference, the :: operator divides the method name from the class or object name.

E.g. System.out::println

In the statement System.out::println, System is a class defined in the java.lang package, out is an instance of the type PrintStream, which is a public and static member field of the System class, and println() is a public method of all instances of the PrintStream class.

```java
package methodreferences;

@FunctionalInterface
interface Display {
    public void getName(String name);
}

public class MethodReferenceInstanceMethod {

    public static void getInfo(String info) {
        System.out.println(info);
    }

```

```java
    public static void main(String[] args) {
        //lambda -> providing implementation of getName(String name)
        Display a = (String s) -> System.out.println(s);
        a.getName("SAM..getName() of Display functional interface is executing");

        /* Method reference : refering  to pre-existing getInfo(String info)
         * of class MethodReferenceInstanceMethod
         * as arguments are same as getName(String name)

        */
        Display a1 = MethodReferenceInstanceMethod::getInfo;
        a1.getName("getInfo() of MethodReferenceInstanceMethod class is executing");

    }
}
```

Problems | Javadoc | Declaration | Console | Outline

<terminated> MethodReferenceInstanceMethod [Java Application] C:\Program Files\Java\jdk-11.0.13\bin\javaw.exe

```
SAM..getName() of Display functional interface is executing
getInfo() of MethodReferenceInstanceMethod class is executing
```

Method References in different category

1. Static Method References
2. Instance method/non-static Method References
3. Constructor References
4. Method References of an arbitrary object of a given type
5. Method References from a super class method.

Note: syntax to write Method Reference(::) Java 8 is different in different cases

Case	Lambda Expression	Method reference equivalent to lambda
Static method	(args) -> ClassName.staticMethodName(args)	ClassName::staticMethodName
Instance Method	(args) -> ObjectName.instanceMethodName(args)	ObjectName::instanceMethodName
Constructor	(args) -> new ClassName(args)	ClassName::new

1. Static Method References

```java
package methodreferences;

@FunctionalInterface
interface Test {
    public boolean checkEvenNo(int x);
}

class Digit {
    public static boolean isEven(int x) {
        boolean result=false;
        if(x%2==0)
            result=true;
        return result;
    }
}

```

```java
public class StaticMethodReference {
    public static void main(String[] args) {

        //*** Using Lambda Expression ***//
        Test a1 = (no) -> {
            boolean result=false;
            if(no%2==0)
                result=true;
            return result;
        };
        System.out.println(a1.checkEvenNo(10));

        //*** Using Method Reference : similar already existing static method***//
        Test a2 = Digit::isEven;
        System.out.println(a2.checkEvenNo(8));
    }
}
```

2. Instance method/non-static Method References

```java
package methodreferences;

@FunctionalInterface
interface B {
    public  void add(int x, int y);
}

class Addition {
    public void sum(int a, int b) {
        System.out.println("The sum is :"+(a+b));
    }
}

public class InstanceMethodReference {

    public static void main(String[] args) {

        Addition addition = new Addition();
        //*** Using Lambda Expression ***//
        B b1 = (a,b) -> System.out.println("The sum is :"+(a+b));
        b1.add(10, 14);

        //*** Using Method Reference ***//
        B b2 = addition::sum;
        b2.add(100, 140);
    }
}
```

3. Constructor Reference

```java
@FunctionalInterface
interface C {
    public Employee getEmployee();
}

@FunctionalInterface
interface D {
    public Employee getEmployee(String name, int age);
}

class Employee {
    String eName;
    int eAge;

    public Employee(){}

    public Employee(String eName, int eAge) {
        this.eName = eName;
        this.eAge = eAge;
    }

    public void getInfo() {
        System.out.println("I am a method of class Employee "+eName+" "+eAge);
    }
}
```

```java
public class ConstructorReference {

    public static void main(String[] args) {

        //*** Using Lambda Expression ***//
        C c1 = () -> new Employee();
        c1.getEmployee().getInfo();

        D d1 = (name,age) -> new Employee(name,age);
        d1.getEmployee("Tony", 34).getInfo();

        //*** Using Method Reference ***//
        C c2 = Employee::new;
        c2.getEmployee().getInfo();

        D d2 = Employee::new;
        d2.getEmployee("Tony", 34).getInfo();
    }
}
```

Class Arrays

https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html

java.lang.Object

    java.util.Arrays

    public class Arrays extends Object

This class is a member of the Java Collections Framework.

Since: 1.2

Modifier and Type	Method	Description
static <T> List<T>	**asList(T... a)**	**Returns a fixed-size list backed by the specified array.**
static float[]	copyOf(float[] original, int newLength)	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
static int[]	copyOf(int[] original, int newLength)	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
static boolean	equals(char[] a, char[] a2)	Returns true if the two specified arrays of chars are *equal* to one another.
static boolean	equals(double[] a, double[] a2)	Returns true if the two specified arrays of doubles are *equal* to one another.
static boolean	equals(float[] a, float[] a2)	Returns true if the two specified arrays of floats are *equal* to one another
static boolean	equals(int[] a, int[] a2)	Returns true if the two specified arrays of ints are *equal* to one another.

Modifier and Type	Method	Description
static void	fill(char[] a, char val)	Assigns the specified char value to each element of the specified array of chars.
static void	fill(double[] a, double val)	Assigns the specified double value to each element of the specified array of doubles.
static void	sort(char[] a)	Sorts the specified array into ascending numerical order.
static void	sort(int[] a)	Sorts the specified array into ascending numerical order.
static IntStream	stream(int[] array)	Returns a sequential IntStream with the specified array as its source.
static DoubleStream	stream(double[] array)	Returns a sequential DoubleStream with the specified array as its source.
static String	toString(float[] a)	Returns a string representation of the contents of the specified array.
static String	toString(int[] a)	Returns a string representation of the contents of the specified array.
static String	toString(Object[] a)	Returns a string representation of the contents of the specified array.

Arrays.asList

1. Using this asList() method of java.util.Arrays class , we can convert from an array to a fixed-size List object.
2. This method acts as a bridge between array-based and collection-based APIs, in combination with Collection.toArray(). The returned list is serializable and implements RandomAccess.
3. we can't modify its length because adding or removing elements is not allowed.
4. However, we can modify single items inside the array. Note that all the modifications we make to the single items of the List will be reflected in our original array

ArrayList(Arrays.asList(array))

1. Similar to the Arrays.asList method, we can use ArrayList<>(Arrays.asList(array))
2. We can create a List out of an array.

```java
package arraypack;

import java.util.ArrayList;

public class ArrayArrayListProgram {
    public static void main(String[] args) {

        String[] stringArray = new String[] { "A", "B", "C", "D" };
        List stringList = Arrays.asList(stringArray);

        System.out.println("Original array\n");
        for(String s:stringArray)
            System.out.print(s);

        System.out.println("\nAdded to List\n"+stringList);


        //modify the first element of stringList:
        //our original array was modified, too.
        //Both the list and the array now contain exactly the same elements in the same order.

        stringList.set(0, "E");
        System.out.println("Original array after modify\n");
        for(String s:stringArray)
            System.out.print(s);
        System.out.println("\nAdded to List\n"+stringList);

```

```java
        //Let's now try to insert a new element to stringList:
        //adding/removing elements to/from the List will throw java.lang.UnsupportedOperationException.

        /*
         * stringList.add("F");
         * for(String s:stringArray)
         * System.out.print(s);
         * System.out.println("\n"+stringList);
         */

        String[] stringArrays = new String[] { "A", "B", "C", "D" };
        List stringLists = new ArrayList<>(Arrays.asList(stringArrays));

        System.out.println("Original array\n");
        for(String s:stringArrays)
            System.out.print(s);
        System.out.println("\nAdded to List\n"+stringLists);

```

```java
        //modify the first element of stringLists:

        /*
         * unlike Arrays.asList(), this is an independent copy of the array,
         * which means that modifying the new list won't affect the original array.
         * Additionally, we have all the capabilities of a regular ArrayList, like adding and removing elements:
         */

        //our original array remains untouched.

            stringLists.set(0, "E");

            System.out.println("Original array after modify \n");
            for(String s:stringArrays)
                System.out.print(s);
            System.out.println("\nAdded to List\n"+stringLists);
```

```java
        //add stringLists:


            stringLists.add("F");

              System.out.println("Original array after add \n");
            for(String s:stringArrays)
            System.out.print(s);
            System.out.println("\nAdded to List\n"+stringLists);
    }

}
```

Note: The type of array must be a Wrapper Class(Integer,Float, etc) in case of primitive data types(int, float,etc) , i.e you can't pass int a[] but you can pass Integer a[]. If you pass int a[], this function will return a List <int a[]> and not List <Integer> , as "autoboxing" doesn't happen in this case and int a[] is itself identified as an object and a List of int array is returned, instead of list of integers , which will give error in various Collection functions .

Reference

MOD -03 -Advanced Java - 4.c Java Collection Framework

Sorting in Collection

We can sort the elements of:

1. String objects
2. Wrapper class objects
3. User-defined class objects

1. If collection elements are of Set type, we can use TreeSet.
2. Collections class provides static methods for sorting the elements of collection.

Method of Collections class for sorting List elements

public void sort(List list): is used to sort the elements of List. List elements must be of Comparable type.

Note: String class and Wrapper classes' implements the Comparable interface. So if you store the objects of string or wrapper classes, it will be Comparable.

Java Comparable Interface

- Java Comparable interface is used to order the objects of user-defined class.
- This interface is found in java.lang package and contains only one method named compareTo(Object obj). It provides single sorting sequence only
- i.e. you can sort the elements on based on single data member only. For example it may be rollno, name, age or anything else.

public int compareTo(Object obj): is used to compare the current object with the specified object.

Java Comparator Interface

- Java Comparator interface is used to order the objects of user-defined class.
- This interface is found in java.util package and contains 2 methods compare(Object obj1,Object obj2)[unimplemented method] and equals(Object element)[Overridden method from Object class].
- It provides multiple sorting sequence i.e. you can sort the elements on the basis of any data member, for example rollno, name, age or anything else.
- public int compare(Object obj1,Object obj2): compares the first object with second object.

method forEach()

- Java provides a new method forEach() to iterate the elements.
- It is defined in Iterable and Stream interface.
- It is a default method defined in the Iterable interface.
- Collection classes which extends Iterable interface can use forEach loop to iterate elements.
- This method takes a single parameter which is a functional interface. So, you can pass lambda expression as an argument.

forEach() Signature in Iterable Interface

```
default void forEach(Consumer<super T>action)
```

```
import java.util.ArrayList;
import java.util.List;
public class ForEachExample {
    public static void main(String[] args) {
            List<String> foodList = new ArrayList<String>();
            foodList.add(“cake");
            foodList.add(“biscuit");
            foodList.add(“cheese");
            System.out.println("------------Iterating by passing lambda expression--------------");
            foodList.forEach(food -> System.out.println(food));
            foodList.forEach(food -> {
                System.out.println(food);
           });
            System.out.println("------------Iterating by passing method reference---------------");
            foodList.forEach(System.out::println);
        }
}
```

4. Method References of an arbitrary object** of a given type

**some instance of the type e.g. a String, or an Integer, or a MyClass

```java
package methodreferences;

import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

public class ArbitraryObjectMethodReference {

public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(15, 33, 59, 24, 40, 2, 19, 25,60);

        //*** Using Anonymous Inner class ***//
        Collections.sort(numbers, new Comparator<Integer>() {
            public int compare(Integer i1,Integer i2) {
                return i1.compareTo(i2);
            }
        });
        System.out.println("************* Using Anonymous Inner class **************");
        numbers.forEach(System.out::println);
```

```java
        //*** Using Lambda Expression ***//
        System.out.println("************ Using Lambda Expression ******************");
        Collections.sort(numbers,(i1,i2) ->i1.compareTo(i2));
        numbers.forEach(System.out::println);

        //*** Using Method Reference ***//
        System.out.println("************ Using Method reference ******************");
        Collections.sort(numbers,(Integer::compareTo));
        numbers.forEach(System.out::println);

        List<String> myString = Arrays.asList("Rishi","Bindu","Anu");
        //*** Using Method Reference ***//
                System.out.println("************ Using Method reference ******************");
                Collections.sort(myString,(String::compareTo));
                myString.forEach(System.out::println);
    }

}
```

5. Method References from a super class method.

```java
package methodreferences;

interface Base {
    public void sayHello();
}

class SuperHello {
    public void superHello(){
        System.out.println("I am inside Super class");
    }
}


class ChildHello extends SuperHello{

    public void childHello() {
        System.out.println("I am inside Sub class  class");
    }
```

```java
    public void testHello() {

        //***Using Method reference :super class method***//
        Base a1= super::superHello;
        a1.sayHello();

        //***Using Lambda Expression :super class method***//
        Base a3= () -> System.out.println("I am inside Super class");
        a3.sayHello();

        //***Using Method reference :current class method***//
        Base a2= this::childHello;
        a2.sayHello();

        //***Using Lambda Expression :current class method***//
        Base a4= () -> System.out.println("I am inside Sub class");
        a4.sayHello();
    }
}
```

```java
public class SuperMethodRef {
    public static void main(String[] args) {

        ChildHello t = new ChildHello();
            t.testHello();
        }

}
```

Problems | Javadoc | Declaration | Console | Outline

<terminated> SuperMethodRef [Java Application] C:\Program Files\Java\jdk-11.0.13\bi

```
I am inside SuperTest class
I am inside SuperTest class
I am inside SubTest class
I am inside SubTest class
```

Thank you . Happy Learning ☺